

Several recent papers have used RL training loops,¹ namely Q-learning, temporal-difference learning, policy gradient, and actor-critic methods, to solve structural economic models at scales where conventional dynamic programming fails.² Throughout this chapter I adopt a unified notation. An MDP is a tuple $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s'|s, a)$ is the transition kernel, $r(s, a)$ is the per-period reward, and $\gamma \in [0, 1)$ is the discount factor.³ A policy $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ maps states to distributions over actions. The value function under π is $V^\pi(s) = \mathbb{E}_\pi [\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s]$, and the action-value function is $Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)}[V^\pi(s')]$. The optimal value function satisfies $V^*(s) = \max_{a \in \mathcal{A}} Q^*(s, a)$.

The canonical structural estimation framework for MDPs was formulated by Rust (1994), whose nested fixed-point (NFXP) algorithm embeds the Bellman equation inside a maximum likelihood estimator. The methods reviewed in this chapter replace the inner fixed-point computation with RL-based approximations.

0.1 Single-Agent Structural Estimation

0.1.1 TD Learning for CCP Estimation

Adusumilli and Eckardt (2022) adapt temporal-difference (TD) learning to estimate the recursive terms that arise in CCP-based estimation, entirely avoiding specification or estimation of transition densities.

The CCP approach requires computing two functions $h : \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}^d$ and $g : \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ that solve the recursive equations

$$h(a, s) = z(s, a) + \gamma \mathbb{E}[h(a', s') \mid a, s], \quad (1)$$

$$g(a, s) = e(a, s) + \gamma \mathbb{E}[g(a', s') \mid a, s], \quad (2)$$

where $e(a, s) = \gamma_E - \ln P(a|s)$ under logit errors (γ_E denoting the Euler constant), and the expectation is over the next-period state-action pair (s', a') given the transition kernel

¹Throughout this chapter, the RL training loop runs entirely inside the econometrician's computational model; the agent never interacts with real economic agents or markets but serves as a numerical method for solving the Bellman equation within a structural estimation procedure (Section ??). There is no execution phase in the usual sense.

²I exclude papers that use neural network function approximation without an RL training mechanism. Inverse reinforcement learning is treated in the sister survey (Rust and Rawat, 2026).

³Several of the papers reviewed here use β for the discount factor, following economics convention. I translate all results to γ for consistency with the RL literature and the rest of this survey.

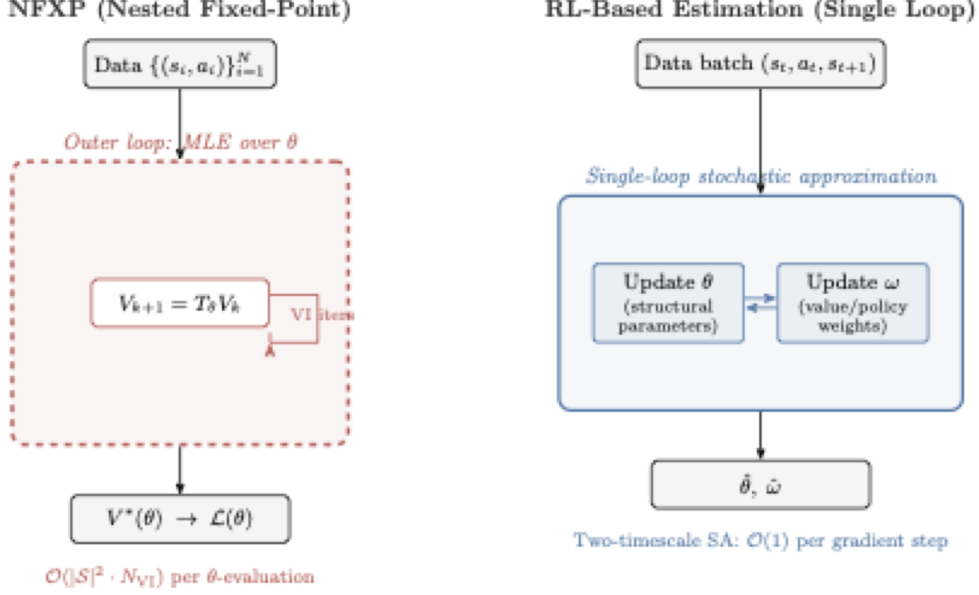


Figure 1: NFXP versus RL-based structural estimation. Top: the nested fixed-point algorithm evaluates the likelihood by solving the Bellman equation to convergence inside each optimizer step. Bottom: single-loop stochastic approximation updates structural parameters θ and value/policy weights ω simultaneously from data batches. The NFXP algorithm is due to Rust (1987).

$P(s'|s, a)$ and the observed policy $P(a|s)$. Both h and g satisfy a Bellman-like recursion under the observed (data-generating) policy, not the optimal policy, so standard TD learning applies directly. Adusumilli and Eckardt (2022) propose two methods.

The first is the linear semi-gradient method.⁴ Approximate $h(a, s) \approx \phi(a, s)^\top w$ where $\phi : \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}^p$ is a vector of basis functions (e.g., polynomials in state variables) and $w \in \mathbb{R}^p$ are the weights to be estimated. The TD(0) fixed-point equation is

$$\mathbb{E} \left[\phi(a, s) (\phi(a, s) - \gamma \phi(a', s'))^\top \right] w = \mathbb{E} [\phi(a, s) z(s, a)]. \quad (3)$$

The sample analog replaces population expectations with averages over the observed panel.

$$\hat{w} = \left(\frac{1}{n(T-1)} \sum_{i=1}^n \sum_{t=1}^{T-1} \phi_{it} (\phi_{it} - \gamma \phi_{i,t+1})^\top \right)^{-1} \left(\frac{1}{n(T-1)} \sum_{i=1}^n \sum_{t=1}^{T-1} \phi_{it} z_{it} \right), \quad (4)$$

⁴The method is called “semi-gradient” because it computes the gradient of only the prediction $\phi(a, s)^\top w$ with respect to w , not the full TD error including the bootstrap target $\phi(a', s')^\top w$. This avoids differentiating through the target but sacrifices guaranteed convergence in some settings.

where $\phi_{it} = \phi(a_{it}, s_{it})$ and $z_{it} = z(s_{it}, a_{it})$. This requires inverting a $p \times p$ matrix, where p is the number of basis functions, making computation trivial in most settings. No transition density estimation is needed, since the method uses only observed sequences of current and next-period state-action pairs.

The second method is approximate value iteration (AVI), which iterates the Bellman-like operator using nonparametric regression. At iteration k , one constructs pseudo-outcomes

$$Y_{it}^{(k)} = z_{it} + \gamma \hat{h}^{(k-1)}(a_{i,t+1}, s_{i,t+1}) \quad (5)$$

and then fits $\hat{h}^{(k)}$ by regressing $Y_{it}^{(k)}$ on (a_{it}, s_{it}) using any machine learning method, including LASSO, random forests, or neural networks.⁵ This is the first DDC estimator compatible with arbitrary ML prediction methods, enabling application to very high-dimensional state spaces.

With $\hat{h}$ and $\hat{g}$ in hand, structural parameters are recovered by pseudo-maximum likelihood estimation (PMLE). For continuous state spaces, [Adusumilli and Eckardt \(2022\)](#) derive a locally robust correction to the PMLE criterion that accounts for the nonparametric first-stage estimation of value terms, restoring $\sqrt{n}$ -convergence of $\hat{\theta}$.⁶ The PMLE score is $m(a, s; \theta, h, g) = \partial_{\theta} \ln \pi(a, s; \theta, h, g)$, where π is the logit choice probability with continuation value $V(a, s) = h(a, s)^{\top} \theta + g(a, s)$. The naive estimator solves $\mathbb{E}_n[m(a, s; \theta, \hat{h}, \hat{g})] = 0$, but with continuous states this moment condition is not orthogonal to the first-stage estimates and converges slower than $\sqrt{n}$. The locally robust moment adds a debiasing correction:

$$\zeta = m(a, s; \theta, h, g) - \lambda(a, s; \theta) \left\{ z(s, a)^{\top} \theta + \gamma e(a', s') + \gamma V(a', s') - V(a, s) \right\}, \quad (6)$$

where $\lambda(a, s; \theta)$ solves a backward recursion that propagates the influence of estimation error through the dynamic structure.⁷ The corrected estimator $\hat{\theta}_{LR}$ solves $\mathbb{E}_n[\zeta_n] = 0$ and is computationally no harder than the naive PMLE, since the correction is constant in θ .

⁵*LASSO* ([Tibshirani, 1996](#)) adds an ℓ_1 penalty to a regression loss, shrinking many coefficients to zero for sparse solutions; *random forests* average many decision trees fit to random subsamples and feature subsets for nonparametric estimation; *neural networks* compose affine transformations with elementwise nonlinearities across multiple layers.

⁶An estimator has $\sqrt{n}$ -convergence if its error shrinks at rate $n^{-1/2}$ where n is the sample size. This is the standard parametric rate; slower rates (e.g., $n^{-1/4}$) indicate efficiency loss from nonparametric first stages.

⁷The term in braces is the temporal-difference error of the continuation value V . The adjoint λ weights this TD error by its marginal impact on the PMLE score. See Online Appendix B.3 of [Adusumilli and Eckardt \(2022\)](#) for the derivation.

Theorem 1 (Adusumilli and Eckardt (2022), Theorem 1). *Under regularity conditions, the linear semi-gradient estimator $\hat{h}$ satisfies $\|\hat{h} - h\|_2 = O_P(n^{-1/2}(T-1)^{-1/2})$, where $\|\cdot\|_2$ denotes the $L^2(P)$ norm.*⁸

Theorem 2 (Adusumilli and Eckardt (2022), Theorem 5). *Under regularity conditions on the ML method used in AVI, the locally robust PMLE estimator $\hat{\theta}_{LR}$ satisfies $\sqrt{n}(\hat{\theta}_{LR} - \theta^*) \xrightarrow{d} \mathcal{N}(0, \Sigma)$ for an explicit variance Σ , even when the state space is continuous.*

Monte Carlo experiments on a dynamic firm entry model with seven structural parameters and five continuous state variables show that the TD-based estimators achieve a 4- to 11-fold reduction in mean squared error compared to CCP estimators using state-space discretization.

For dynamic discrete games, the method extends naturally. Standard CCP-based estimation of games requires integrating out other players’ actions, which becomes intractable with many players or continuous states. TD learning avoids this entirely, since it works directly with the joint empirical distribution of states and their successors. The “integrating out” is done implicitly within sample expectations.

0.1.2 Policy Gradient for DDC Estimation

Hu and Yang (2025) combine policy gradient methods with the Simulated Method of Moments (SMM) to estimate DDCs, with particular focus on models with unobserved state variables.

The outer loop is SMM.⁹ Define a vector of data moments $\mathbf{M}_d$ computed from the observed panel. For candidate structural parameters θ and transition parameters ξ , simulate the model to produce simulated moments $\mathbf{M}_s(\theta, \xi)$. The estimator minimizes

$$(\hat{\theta}, \hat{\xi}) = \arg \min_{\theta, \xi} (\mathbf{M}_d - \mathbf{M}_s(\theta, \xi))^\top \mathbf{W} (\mathbf{M}_d - \mathbf{M}_s(\theta, \xi)), \quad (7)$$

where $\mathbf{W}$ is a positive definite weight matrix. Computing $\mathbf{M}_s(\theta, \xi)$ requires solving for the optimal policy under (θ, ξ) , which is the inner-loop problem.

The inner loop parametrizes the choice probability directly as a logistic function of

⁸The $L^2(P)$ norm is $\|f\|_2 = (\int f(x)^2 dP(x))^{1/2}$, measuring average squared deviation under the probability measure P . This is the natural norm for mean-squared-error analysis.

⁹SMM (Gourieroux et al., 1993) estimates structural parameters by matching moments from simulated data to moments from observed data, avoiding direct evaluation of the likelihood function.

state variables. For a binary choice $J_t \in \{0, 1\}$, the general form is $\Pr(J_t = 1 \mid \mathbf{X}_t; \boldsymbol{\gamma}(\theta)) = \text{logistic}(\mathbf{X}_t \boldsymbol{\gamma}(\theta))$, where $\boldsymbol{\gamma}(\theta)$ are policy parameters that depend on the structural parameters.¹⁰ In their application to a Rust bus engine model with unobserved bus condition S_t^* , this takes the form

$$\Pr(J_t = 1 \mid X_t, S_t^*, t; \boldsymbol{\gamma}) = \frac{\exp(\gamma_0 + \gamma_1 t + \gamma_2 X_t + \gamma_3 S_t^*)}{1 + \exp(\gamma_0 + \gamma_1 t + \gamma_2 X_t + \gamma_3 S_t^*)}, \quad (8)$$

where t enters the index directly to capture time-varying replacement incentives. The policy parameters are updated by REINFORCE-style gradient ascent, applying the policy gradient theorem (Sutton et al., 1999):

$$\nabla_{\boldsymbol{\gamma}} V(\boldsymbol{\gamma}) = \mathbb{E} \left[\sum_{t=0}^T \nabla_{\boldsymbol{\gamma}} \log \pi_{\boldsymbol{\gamma}}(J_t \mid X_t, S_t^*, t) Q^{\pi_{\boldsymbol{\gamma}}}(X_t, S_t^*, J_t) \right], \quad (9)$$

where $Q^{\pi_{\boldsymbol{\gamma}}}$ is the action-value function under the current policy, estimated by Monte Carlo returns from forward-simulated trajectories.

The main contribution is handling unobserved state variables. When state variables are only partially observed, the policy in (8) is parametrized as a function of both X_t and S_t^* , and the algorithm forward-simulates trajectories of both observed and unobserved variables.

Building on the nonparametric identification results of Hu and Shum (2012), the outer-loop SMM targets moments from five consecutive periods of observed data, which suffice to separately identify the structural parameters θ and transition parameters ξ without requiring the econometrician to observe S_t^* . No discretization of continuous unobserved states is needed; the same algorithm handles both discrete and continuous unobserved heterogeneity.

For each candidate (θ, ξ) in the outer minimization (7), the inner loop runs policy gradient until convergence, producing optimal policy parameters $\boldsymbol{\gamma}^*(\theta, \xi)$. These are used to simulate data and compute $\mathbf{M}_s(\theta, \xi)$.

On an extended Rust bus engine model with a continuous unobserved bus condition following an AR(1) process, estimates of seven structural parameters are centered around their true values across 400 simulations. On a discrete-unobservable variant, the

¹⁰The linear index can be replaced by higher-order terms of $\mathbf{X}_t$ or deep neural networks; the method requires only that the gradient $\nabla_{\boldsymbol{\gamma}} \log \pi_{\boldsymbol{\gamma}}$ has a closed form.

method matches the precision of Arcidiacono and Miller (2011)’s two-step EM algorithm at comparable computation times, though the advantage diminishes as more inner-loop iterations are used for precision.

0.2 Dynamic Oligopoly and Strategic Interaction

Dynamic oligopoly models combine game theory and dynamic programming: firms choose actions strategically while anticipating competitors’ strategies, and the state space grows combinatorially in the number of firms.

0.2.1 Q-Learning in Dynamic Procurement Auctions

Asker et al. (2020) develop a computational framework for analyzing dynamic procurement auctions with serially correlated asymmetric information. Their approach builds on the Experience-Based Equilibrium (EBE) concept of Fershtman and Pakes (2012), which computes equilibria by simulating industry trajectories and updating strategies toward best responses. EBE evaluates values only on recurrently visited states rather than the full state space, making it feasible for large state spaces. While Fershtman and Pakes (2012) use a stochastic approximation algorithm to update continuation values from simulated industry trajectories, Asker et al. (2020) add explicit value-function updates via stochastic approximation.

The model is a repeated first-price sealed-bid auction with two firms. Each firm i maintains a private inventory state $\omega_{i,t}$ (stock of unharvested timber, in their application). The state evolves endogenously, as winning an auction increases inventory while harvesting depletes it. Each firm’s private state is not observed by its competitor except at periodic revelation events.

The key computational innovation is the use of Q-learning to compute equilibrium strategies. Each firm i maintains a Q-function $Q_i : \mathcal{S}_i \times \mathcal{A}_i \rightarrow \mathbb{R}$, where $\mathcal{S}_i$ encodes firm i ’s information set (its own inventory, beliefs about the competitor’s inventory, public history) and $\mathcal{A}_i$ is its action set (participation decision and bid level). The Q-function satisfies

$$Q_i(s, a) = \mathbb{E} \left[r_i(s, a, a_{-i}) + \gamma \max_{a' \in \mathcal{A}_i} Q_i(s', a') \mid s, a \right], \quad (10)$$

where $r_i(s, a, a_{-i})$ is firm i ’s per-period profit given the state, its own action a , and the

competitor's action a_{-i} , and the expectation is taken over the competitor's strategy and the stochastic transitions.

Firms update Q-values using sample averaging.

$$Q_i(s, a) \leftarrow Q_i(s, a) + \frac{1}{h_k(s, a)} \left[r_i + \gamma \max_{a' \in \mathcal{A}_i} Q_i(s', a') - Q_i(s, a) \right], \quad (11)$$

where $h_k(s, a)$ is the number of times state-action pair (s, a) has been visited.¹¹ The equilibrium computation proceeds iteratively, with firms simultaneously updating their Q-functions based on simulated play against each other's current strategies. Strategies are derived from Q-values using an ε -greedy rule or Boltzmann exploration.¹²

Asker et al. (2020) add a boundary consistency condition to the EBE concept that restricts behavior at the boundary of the recurrent state class, reducing the multiplicity of equilibria. Their numerical analysis reveals that information sharing between firms can, through increased precision of beliefs about competitor states, induce firms to spend more time in states where competition is less intense. The dynamic RL-computed equilibrium yields qualitatively different predictions from both static analysis and myopic ($\gamma = 0$) benchmarks. With dynamics, information sharing decreases average bids and increases average profits, while the myopic benchmark shows negligible effects.

The limitation of this approach is the tabular representation; the Q-function is stored as a lookup table over discretized states and actions, restricting applicability to models with moderate state-space dimension.

0.2.2 TD Learning for Merger Analysis with Innovation

Hollenbeck (2019) uses RL to solve a dynamic oligopoly model with endogenous mergers, entry, exit, and quality investment. The model extends the Ericson-Pakes framework to study how horizontal mergers affect innovation incentives.

The industry state is $\Omega = (\omega_1, \dots, \omega_n)$ where $\omega_i \in \{1, \dots, \omega_{\max}\}$ is firm i 's product quality. Firms produce differentiated goods and compete in prices (Bertrand competition with logit demand). In each period, firms simultaneously choose investment levels,

¹¹This sample-averaging rule ($\alpha_k = 1/h_k$) is equivalent to maintaining the running mean of observed returns, as distinct from fixed- α Q-learning which gives exponentially decaying weight to older observations. The update is applied for all actions, including counterfactual actions not taken, using the observed state transition.

¹² ε -greedy selects the greedy action $\arg\max_a Q(s, a)$ with probability $1 - \varepsilon$ and a uniformly random action otherwise. Boltzmann exploration selects action a with probability $\propto \exp(Q(s, a)/\tau)$ where $\tau > 0$ is a temperature parameter.

entry/exit decisions, and potentially initiate merger negotiations.

Each firm i computes its continuation value $V_i(\Omega)$ from the industry state. Because the state space is the product of all firms' quality levels plus industry structure (number of active firms, recent mergers), exact dynamic programming is infeasible for industries with more than two or three firms. [Hollenbeck \(2019\)](#) instead uses temporal-difference learning to estimate values from simulated industry trajectories.

The value function update for firm i is¹³

$$V_i(\Omega) \leftarrow V_i(\Omega) + \alpha [\Pi_i(\Omega, \mathbf{a}) + \gamma V_i(\Omega') - V_i(\Omega)], \quad (12)$$

where $\Pi_i(\Omega, \mathbf{a})$ denotes firm i 's per-period profit given industry state Ω and the joint action profile $\mathbf{a}$ (investment, entry/exit, merger decisions),¹⁴ γ is the discount factor, and Ω' is the realized next-period state. The algorithm uses ε -decreasing exploration to prevent convergence to locally suboptimal equilibria.

The equilibrium computation follows the Pakes-McGuire iterative scheme.¹⁵ The algorithm simulates long industry histories, updates each firm's value function via (12), re-derives best-response strategies from updated values, and repeats.

The central finding is that horizontal mergers, while reducing static consumer surplus in the short run, create a strong incentive for entry and investment. Firms enter with negative static profits because the prospect of a lucrative buyout justifies the initial investment. The result is substantially higher long-run innovation and consumer welfare with mergers than without. This finding reverses the standard static antitrust prediction and can only emerge in a dynamic model where firms are forward-looking.

Two related papers merit brief mention. [Lomys and Magnolfi \(2024\)](#) develop structural estimation methods for strategic settings where agents use learning algorithms (specifically regret-minimizing rules) rather than playing a fixed equilibrium. They impose an "asymptotic no-regret" condition as a minimal rationality requirement and derive identification results for payoff parameters. [Covarrubias \(2022\)](#) uses deep RL to study

¹³This stochastic approximation update, introduced by [Pakes and McGuire \(1994\)](#) for dynamic oligopoly computation, is closely related to temporal-difference learning in the RL literature. The original algorithm uses visit-count averaging ($\alpha_k = 1/k$ where k counts visits to each state) rather than a fixed learning rate.

¹⁴I use Π_i for firm i 's per-period profit to avoid confusion with policy π .

¹⁵The Pakes-McGuire algorithm ([Pakes and McGuire, 1994](#)) computes Markov perfect equilibria by iterating: (1) given current value functions, compute best-response strategies; (2) given strategies, update value functions via simulation. Convergence is not guaranteed but works well in practice.

oligopolistic pricing in a New Keynesian framework, representing firms’ pricing policies as neural networks $\pi_\phi(a|s)$. The method uncovers multiple equilibria ranging from competitive to collusive pricing.¹⁶

0.3 Auction Equilibria and Mechanism Design

Closed-form equilibrium bidding strategies exist only for narrow families of valuation distributions and auction formats, and numerical methods scale poorly with the number of bidders and items.

0.3.1 RL for Sequential Price Mechanisms

Brero et al. (2021) use RL to design optimal sequential price mechanisms (SPMs), a class of indirect auction mechanisms where agents are approached in sequence and offered menus of items at posted prices. SPMs generalize both serial dictatorship and posted-price mechanisms and essentially characterize all strongly obviously strategyproof (SOSP) mechanisms (Pycia and Troyan, 2023).¹⁷

The mechanism design problem is formulated as a partially observable Markov decision process (POMDP).¹⁸ The state at round t includes the set of remaining items $\rho_t^{\text{items}} \subseteq [m]$, remaining agents $\rho_t^{\text{agents}} \subseteq [n]$, the partial allocation $\mathbf{x}_t$, and the agents’ (unobserved) valuation functions. The action $a_t = (i_t, \{p_j^t\}_{j \in \rho_{t-1}^{\text{items}}})$ specifies which agent to visit next and what prices to offer. The observation is the agent’s purchase decision, from which the mechanism can update beliefs about valuations. The reward is the objective function evaluated at the final allocation:

$$r = g(\mathbf{x}_T, \boldsymbol{\tau}_T; \mathbf{v}), \quad (13)$$

where g can be social welfare $\sum_i v_i(x_i)$, revenue $\sum_i \tau_i$, or max-min fairness $\min_i v_i(x_i)$.

A key theoretical result establishes when adaptive mechanisms outperform static ones.

¹⁶The most prominent example of algorithmic collusion is Calvano et al. (2020), who showed that independent Q-learning agents in a repeated Bertrand pricing game learn to sustain supra-competitive prices and punish deviators without explicit communication. This result and its implications for competition policy are treated in the companion thesis chapter (Rawat, 2026).

¹⁷A mechanism is strategyproof if truthful reporting is a dominant strategy. It is obviously strategyproof if this dominance is apparent even to boundedly rational agents; strongly obviously strategyproof (SOSP) adds that dominance holds at every information set (Pycia and Troyan, 2023).

¹⁸In a POMDP, the agent cannot directly observe the full state. It maintains a belief distribution over possible states, updated via Bayes’ rule as new observations arrive. This captures the mechanism designer’s uncertainty about bidder valuations.

Theorem 3 (Brero et al. (2021), Propositions 1–4, informal). *Each feature of adaptive mechanisms is necessary for welfare optimality, even in simple settings. Personalized prices are needed with one item and two i.i.d. agents (Proposition 1); adaptive prices are needed with two identical items and three i.i.d. agents (Proposition 2); adaptive ordering is needed with six agents whose valuations are correlated (Proposition 3); both adaptive prices and ordering are needed with four agents whose valuations are independently but non-identically distributed (Proposition 4).*

The policy maps from a sufficient statistic of the observation history to actions. Brero et al. (2021) show that this statistic can be represented compactly. The set of remaining items and agents suffices for independent valuations, while the full allocation matrix is needed for correlated valuations. They train the mechanism policy using Proximal Policy Optimization (PPO),¹⁹ which handles the discrete action space (agent selection) and continuous action space (price setting) of the POMDP.

Experimental results show that the learned SPMs achieve near-optimal welfare across settings with up to 20 agents and 5 items (with similar results noted for up to 30 of each), significantly outperforming static pricing benchmarks. The improvement is largest when agent valuations are correlated, since adaptive prices allow the mechanism to infer information about remaining agents from earlier purchases.

The limitation is that the POMDP formulation requires knowledge of the prior distribution over valuations, which in practice must be estimated from data. The method also does not scale easily to very large numbers of items due to the combinatorial action space.

0.3.2 Fitted Policy Iteration for Combinatorial Auctions

Ravindranath et al. (2024) address revenue-maximizing mechanism design for combinatorial auctions with multiple items and strategic bidders. Their innovation is integrating differentiable auction structure into a fitted policy iteration framework, enabling analytical gradient computation where standard RL methods struggle with high variance.

The mechanism visits agents one at a time in sequence. Each agent i , upon being visited, selects a bundle of items from those still available, given posted prices. Valua-

¹⁹Proximal Policy Optimization (Schulman et al., 2017) is a policy gradient algorithm that constrains each update to a trust region, preventing large destabilizing policy changes. It is described in Chapter 2.

tions are drawn once from distributions V_i and remain fixed throughout the mechanism. Complementarities arise from the structure of the valuation function over bundles, not from dynamic evolution. The MDP state at step t is $s_t = (i_t, S_t)$, where i_t is the current bidder and S_t is the set of remaining items.

The mechanism’s policy maps from state to a price vector over available items. The key technical contribution is making the auction clearing differentiable. In a standard auction, the allocation is an argmax over bids (non-differentiable), and payments depend discontinuously on the allocation. Ravindranath et al. (2024) replace the hard bundle selection with a softmax relaxation,²⁰ enabling analytical gradient computation through the mechanism. The actor loss is the negative expected revenue, and gradients flow directly through the softmax-relaxed allocation and payment rules. The paper explicitly avoids REINFORCE-style estimators, noting that analytical gradients overcome the sample inefficiency and high variance of score-function methods.

The method follows fitted policy iteration (Bertsekas and Tsitsiklis, 1996), alternating between evaluating the current policy (computing expected revenue) and improving it via gradient ascent on the actor loss. This differs from model-free RL approaches (PPO, SAC) that the paper uses as baselines.

Experiments on settings with additive and combinatorial valuations show that the learned mechanisms achieve up to 13% higher revenue than item-wise Myerson optimal auctions, with the largest gains in combinatorial settings where bundle complementarities make item-wise pricing suboptimal. Standard RL baselines (PPO, SAC) are also outperformed, confirming the advantage of exploiting differentiable structure.²¹

0.4 Simulation Study: DDC Estimation at Scale

The test bed is a multi-component extension of the Rust (1987) bus engine replacement model. In the original formulation a maintenance superintendent observes discretized mileage $m \in \{0, \dots, M-1\}$ and makes a binary keep-or-replace decision, facing running cost $c(s; \theta) = \theta_1 x + \theta_2 x^2$ with $x = m/M$, replacement cost RC , and Type I extreme value additive errors that yield logit conditional choice probabilities. We extend the model to K independent wear components, each evolving in $\{0, \dots, M-1\}$, with aggregate normalized

²⁰The softmax relaxation replaces the hard allocation $\operatorname{argmax}_i b_i$ with a soft allocation $\exp(b_i/\tau)/\sum_j \exp(b_j/\tau)$, which is differentiable and approaches the hard allocation as $\tau \rightarrow 0$.

²¹DDPG was also tested but found to be unstable. DQN is not applicable to continuous price-setting.

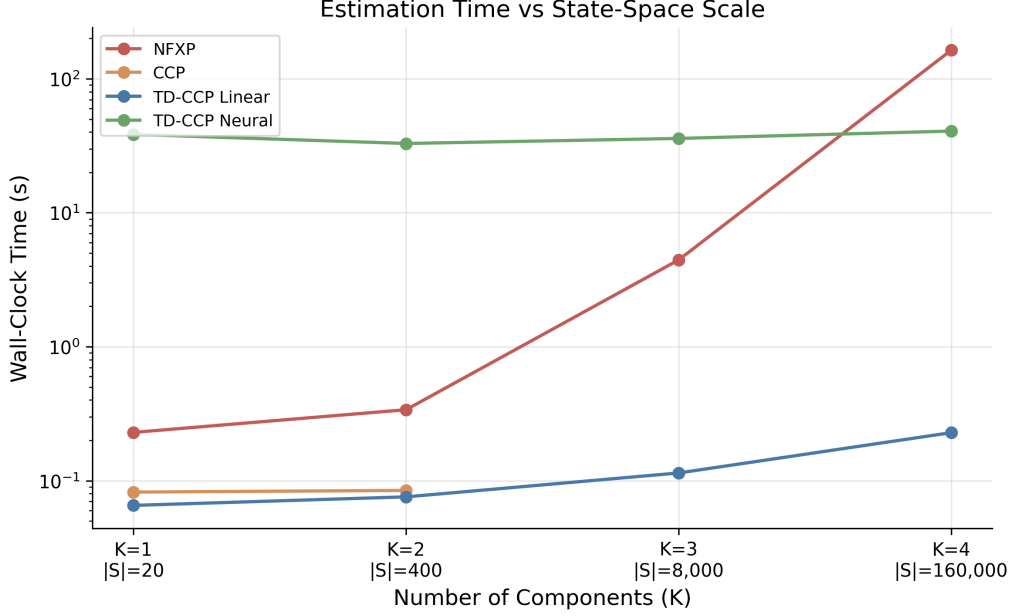


Figure 2: Wall-clock estimation time versus state-space scale for the four DDC estimators. Vertical axis is log-scaled. Each point is the mean over 10 seeds.

wear $x(s) = \sum_k m_k/M$ entering the same cost function. The state space is $|\mathcal{S}| = M^K$, so increasing K produces a controlled scaling experiment in which the data-generating process is identical across scales but the computational burden grows exponentially.

We compare four estimation methods on a multi-component bus engine replacement problem (Rust, 1987) with $K \in \{1, 2, 3, 4\}$ independent wear components, producing state spaces from 20 to 160,000 states. Panel data consist of $N = 500$ agents observed for $T = 100$ periods. The four methods are NFXP (nested fixed-point MLE), CCP (Hotz-Miller inversion), TD-CCP Linear (semi-gradient TD with polynomial basis), and TD-CCP Neural (approximate value iteration with a two-layer MLP), where the two TD-CCP variants follow Adusumilli and Eckardt (2022).²² Each configuration is replicated across 10 seeds with the PyTorch optimizer also seeded; Table 1 and Figure 2 report means with seed-level standard errors.

NFXP wall-clock grows by roughly three orders of magnitude across the four scales,

²²The TD-CCP variants implemented here are simplified relative to Adusumilli and Eckardt (2022). We omit the locally robust PMLE correction (Theorem 5 of that paper, the construction that yields $\sqrt{n}$ -consistency); the reported point estimates are for the simpler plug-in PMLE and do not inherit the paper’s $\sqrt{n}$ -convergence guarantee. We also reformulate the bootstrap target by decomposing $EV(s; \theta)$ into θ -linear pieces and learning each piece separately, rather than fitting $h(a, s)$ directly as in the paper; the two are algebraically related under our binary action with one-period reset, but the change is undocumented in the original. Finally, the marketed advantage of TD-CCP that “no transition density is needed” is qualified in our implementation: the PMLE step still evaluates $v_0 = -c + \gamma P_{\text{keep}} \widehat{EV}$ using the empirical P_{keep} assumed known under the Rust convention. A fully density-free variant that replaces $P_{\text{keep}} \widehat{EV}$ by an out-of-sample target average is left for future work.

Table 1: DDC estimation results across state-space scales. Rows show method, number of components K , state-space size $|\mathcal{S}|$, mean wall-clock time (seconds), RC bias, RC RMSE, and root mean squared error for θ_1 and θ_2 . Standard errors in adjacent columns; means over 10 seeds, dashes indicate method failure (sparse state coverage).

Method	K	$ \mathcal{S} $	Time (s)	RC Bias		RC RMSE		θ_1 RMSE		θ_2 RMSE	
				Mean	SE	Value	SE	Value	SE	Value	SE
NFXP	1	20	0.2	+0.053	0.027	0.097	0.015	0.307	0.055	0.495	0.111
CCP	1	20	0.1	+0.055	0.027	0.099	0.015	0.316	0.055	0.510	0.112
TD-CCP Linear	1	20	0.1	-0.087	0.017	0.101	0.016	0.255	0.041	0.296	0.044
TD-CCP Neural	1	20	38.4	+0.054	0.026	0.094	0.013	0.260	0.054	0.423	0.082
NFXP	2	400	0.3	+0.027	0.028	0.087	0.018	0.270	0.061	0.344	0.077
CCP	2	400	0.1	+0.089	0.027	0.120	0.021	0.527	0.068	0.797	0.089
TD-CCP Linear	2	400	0.1	-0.171	0.021	0.182	0.021	0.131	0.027	0.888	0.060
TD-CCP Neural	2	400	32.9	+0.012	0.030	0.090	0.021	0.247	0.065	0.334	0.067
NFXP	3	8,000	4.4	+0.011	0.014	0.044	0.009	0.236	0.046	0.364	0.081
CCP	3	8,000	—	—	—	—	—	—	—	—	—
TD-CCP Linear	3	8,000	0.1	-0.246	0.011	0.248	0.010	0.125	0.022	1.178	0.071
TD-CCP Neural	3	8,000	35.9	+0.007	0.017	0.051	0.009	0.215	0.050	0.318	0.081
NFXP	4	160,000	163.5	-0.034	0.017	0.061	0.012	0.153	0.027	0.143	0.024
CCP	4	160,000	—	—	—	—	—	—	—	—	—
TD-CCP Linear	4	160,000	0.2	-0.328	0.014	0.331	0.014	0.226	0.028	1.103	0.027
TD-CCP Neural	4	160,000	40.7	-0.031	0.019	0.066	0.011	0.258	0.042	0.236	0.039

reflecting the cost of repeated value iteration inside each likelihood evaluation. CCP becomes infeasible beyond $K=2$ due to sparse state coverage in the panel data. TD-CCP Neural maintains near-constant runtime across all K levels with RC RMSE comparable to NFXP, consistent with the AVI approach of [Adusumilli and Eckardt \(2022\)](#). TD-CCP Linear runs in well under a second at all scales but suffers from basis misspecification at higher K , with RC RMSE rising monotonically in K ; see Table 1 and Figure 2.

References

- Karun Adusumilli and Dita Eckardt. Temporal-difference estimation of dynamic discrete choice models. *arXiv preprint arXiv:2209.15174*, 2022.
- Peter Arcidiacono and Robert A. Miller. Conditional choice probability estimation of dynamic discrete choice models with unobserved heterogeneity. *Econometrica*, 79(6): 1823–1867, 2011.
- John Asker, Chaim Fershtman, Jihye Jeon, and Ariel Pakes. A computational framework for analyzing dynamic auctions: The market impact of information sharing. *The RAND Journal of Economics*, 51(3):805–839, 2020.
- Dimitri P. Bertsekas and John N. Tsitsiklis. *Neuro-Dynamic Programming*. Athena Scientific, Belmont, MA, 1996.
- Gianluca Brero, Alon Eden, Matthias Gerstgrasser, David C. Parkes, and Duncan Rheingans-Yoo. Reinforcement learning of sequential price mechanisms. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 13662–13670, 2021.
- Emilio Calvano, Giacomo Calzolari, Vincenzo Denicolò, and Sergio Pastorello. Artificial intelligence, algorithmic pricing, and collusion. *American Economic Review*, 110(10): 3267–3297, 2020.
- Matias Covarrubias. Dynamic oligopoly and monetary policy: A deep reinforcement learning approach. Job Market Paper, New York University, 2022.
- Chaim Fershtman and Ariel Pakes. Dynamic games with asymmetric information: A framework for empirical work. *The Quarterly Journal of Economics*, 127(4):1611–1661, 2012.
- Christian Gourieroux, Alain Monfort, and Eric Renault. Indirect inference. *Journal of Applied Econometrics*, 8(S1):S85–S118, 1993.
- Brett Hollenbeck. Horizontal mergers and innovation in concentrated industries. *Quantitative Marketing and Economics*, 2019.
- Yingyao Hu and Matthew Shum. Nonparametric identification of dynamic models with unobserved state variables. *Journal of Econometrics*, 171(1):32–44, 2012.

- Yingyao Hu and Fangzhu Yang. Estimation of dynamic discrete choice models with unobserved state variables using reinforcement learning. *Working Paper, Johns Hopkins University*, 2025.
- Nikolay Lomys and Luca Magnolfi. Estimation of games under no regret: Structural econometrics for ai. Working Paper NET Institute Working Paper No. 24-05, Social Science Research Network (SSRN), 2024. Available at SSRN: <https://ssrn.com/abstract=4717195> or <http://dx.doi.org/10.2139/ssrn.4717195>.
- Ariel Pakes and Paul McGuire. Computing Markov-perfect Nash equilibria: Numerical implications of a dynamic differentiated product model. *RAND Journal of Economics*, 25(4):555–589, 1994.
- Marek Pycia and Peter Troyan. A theory of simplicity in games and mechanism design. *Econometrica*, 91(4):1495–1526, 2023.
- Sai Srivatsa Ravindranath, Zhe Feng, Di Wang, Manzil Zaheer, Aranyak Mehta, and David C. Parkes. Deep reinforcement learning for sequential combinatorial auctions. Submitted to ICLR 2025, 2024. Available at <https://openreview.net/forum?id=SVd9Ffcdp8>.
- Pranjal Rawat. Designing auctions when algorithms learn to bid. *Working Paper*, 2026.
- John Rust. Optimal replacement of gmc bus engines: An empirical model of harold zurcher. *Econometrica*, 55(5):999–1033, 1987.
- John Rust. Structural estimation of Markov decision processes. In Robert F. Engle and Daniel McFadden, editors, *Handbook of Econometrics*, volume 4, chapter 51, pages 3081–3143. Elsevier, 1994.
- John Rust and Pranjal Rawat. Structural econometrics and inverse reinforcement learning: Inferring preferences and beliefs from human behavior. Working Paper, Georgetown University, 2026.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Richard S. Sutton, David A. McAllester, Satinder P. Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in Neural Information Processing Systems*, 12, 1999.

Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society, Series B*, 58(1):267–288, 1996.